



Nombre:

Grupo:

Pregunta 1:

Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

- [] Si la clase ClassA tiene el siguiente atributo:
*map<string,ClassB> *atributo;* entonces necesariamente la relación entre ClassA y ClassB es de asociación.
- [] Una matriz dispersa y una cola con prioridad pueden tener en común el contenedor sobre el que se montan ambas estructuras de datos.
- [] Si esta sentencia es válida:
int nuevoValor = 7;
it=mieedd.find(clave);
(*it).second[i] = &nuevoValor;
Entonces *mieedd* puede tener esta definición:
map <int, vector<int> > mieedd;
- [] Un vector estático puede implementar eficientemente a pilas y colas estáticas.
- [] Se pueden listar los datos de un árbol AVL o ABB en orden inverso a su definición sin necesidad de añadir un

- puntero al padre.
- [] No es posible que un árbol AVL tenga un nodo hoja a una profundidad 4 y otra a profundidad 8.
- [] El recorrido en profundidad de un grafo usa normalmente una cola mientras que en anchura su implementación es con pila o recursiva.
- [] Dada la siguiente definición, la clase Fecha necesita la definición del operador>
multimap<Fecha, Reserva, greater<Fecha> > reservas;
- [] En el proceso de inserción de un dato en un árbol B se puede repartir la carga de datos con nodos hermanos en el caso de que dicho dato no quepa en el nodo asignado.
- [] Dado un nodo de un quadtree cuya codificación es 00-11-01-00, se puede saber exactamente las coordenadas del área al que hace referencia.

Pregunta 2:

En el control de acceso a una tienda Web se desea mantener una estructura de datos con la lista de usuarios que han entrado en el sistema. Dicha lista se almacena en un vector dinámico pero para acelerar las búsquedas se usan dos árboles AVL auxiliares, ordenados por nombre de usuario y por fecha/hora de último acceso (en un long con formato AAAAMMDDHHMMSS). Las clases almacenadas en el vector y en los dos árboles AVE son las siguientes:

```
/* EEDD vector */
class Usuario{
public:
    string nombre;
    long ultimoAcceso;
    string e-mail;
};

/* EEDD AVL usuario por nombre */
class UsuarioPorNombre{
public:
    string nombre;
    Usuario *usuario;

    bool operator<(const UsuarioPorNombre& u){
        return nombre < u.nombre;
    }
};
```

```
/* EEDD AVL Usuario por fecha último acceso */
class UsuarioPorAcceso{
public:
    long ultimoAcceso;
    Usuario *usuario;

    bool operator<(const UsuarioPorAcceso& u){
        return ultimoAcceso < u.ultimoAcceso;
    }
};
```

Mostrar las estructuras de datos descritas durante las siguientes inserciones (agrupar pasos siempre que no se dificulte la lectura del ejercicio):
I(Usuario("pruiz", 20161205101402, "pruiz@gmail.com")), *I(Usuario("alopez", 20170103122000, "alopez@gmail.com"))*,
I(Usuario("igomez", 20161110091523, "igomez@hotmail.com")), *I(Usuario("mhernandez", 20161122173045, "mhernandez@yahoo.es"))*,
I(Usuario("yperez", 20170120193215, "yperez@hotmail.com"))

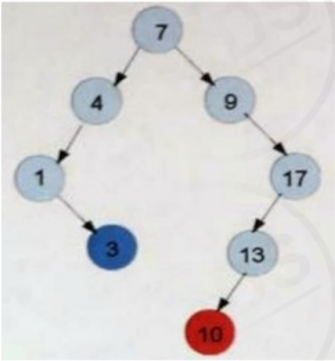


Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

Pregunta 3:

- (a) Implementar el constructor de la clase ListaDEnlazada para que cree un instancia de una lista doblemente enlazada a partir de los datos de una simplemente enlazada, teniendo en cuenta que a priori no son clases amigas.
- template <class T>
ListaDEnlazada<T>::ListaDEnlazada(const
ListaEnlazada &l);
- (b) Implementar la función de inserción de un dato en un árbol ABB:
- bool ABB<T>::insertar(const T &dato);
- Debe insertar el dato normalmente pero que además se pide que devuelva el valor TRUE cuando al insertar el dato se genere una rama de 4 o más nodos seguidos tipo lista, es decir, que a lo largo del camino o cuando se inserta el nuevo dato se encuentren 4 o más nodos sin

nodos hermanos. En el siguiente ejemplo la inserción de 3 no genera este aviso, pero la inserción de 10 sí, al generar una rama de cuatro nodos sin hermanos: 9→17→13→10.



Pregunta 4:

Los propietarios de la tienda Web descrita en la pregunta 2 han decidido rediseñar el sistema de una manera más ambiciosa para hacer estudios de mercado usando técnicas de BigData. Para ello han implementado un potente sistema de consultas representado por el diagrama UML adjunto:

La clase **Usuario** se ha ampliado con información adicional (dirección, localidad), incluyendo una lista histórica de los productos que ha comprado el usuario.

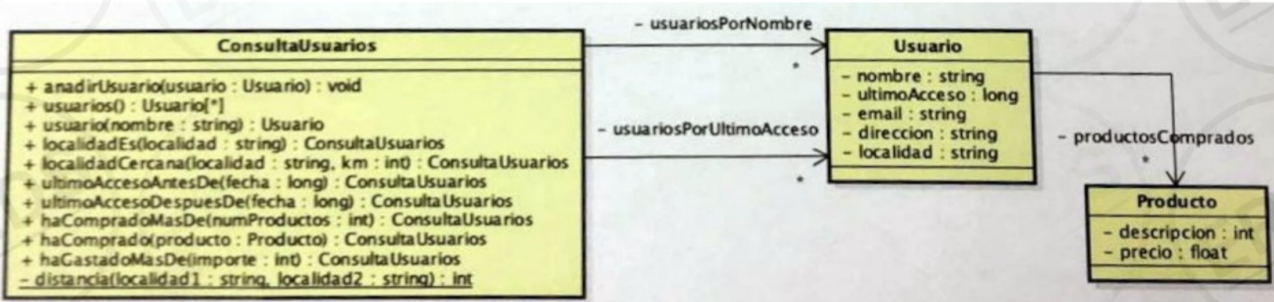
La clase **ConsultaUsuario** contiene un conjunto de usuarios que pueden accederse de forma eficiente por dos criterios diferentes: nombre de usuario y fecha de último acceso. Esta clase contiene una serie de operaciones que permiten filtrar los usuarios por distintos criterios, devolviendo otro objeto de tipo ConsultaUsuario con el resultado. Las operaciones son autoexplicativas así que tan solo añadiremos un par de algunas aclaraciones adicionales:

- *usuarios()*: Devuelve una estructura de datos con todos los usuarios de la consulta.
- *localidadCercana()*: Devuelve una consulta con todos los usuarios que viven a menos de una distancia dada (km) de otra localidad. Para implementar esta operación usaremos la operación estática privada *distancia()*, que daremos por implementada.

Implementar las tres clases del diagrama. Implementar las consultas de Consulta Usuarios de la manera más eficiente posible y con las mínimas redundancias. Una vez terminado el sistema, debe permitir consultas encadenadas como esta:

```
// Obtener los usuarios de Jaén y 20 km. alrededor que hayan accedido este año al sistema
// y hayan gastado más de 1000 euros en total desde que se dieron de alta
// todos es de tipo ConsultaUsuarios y contiene todos los usuarios del sistema

vector<Usuario *> resultado = todos .hanGastadoMasDe(1000) .localidadCercana("Jaén", 25)
                                .ultimoAccesoDespuesDe(Fecha ("1/1/2017").devolverLong()).usuarios();
```



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional